

Universidad Autónoma de Yucatán

Facultad de Matemáticas

Asignatura:

Programación Orientada a Objetos

Proyecto: UniRide

Hood Code Department:

Moguel Miranda, Nicolas

Uicab Gutierrez, Gadiel Abdias

Profesor:

Lic. Miguel Ángel Canul Chin

Mérida, Yucatán. a 21 de Junio de 2026



1. Introducción y Objetivos

1.1 Contexto y Definición del Problema

El medio de transporte es principalmente uno de los problemas más frecuentes a la hora de desplazarse hacia la universidad. Para ello, el gobierno ha implementado rutas de transporte público para mitigar este problema; esto implica que los estudiantes suelen usar esta facilidad, así ocurre una liberación de la carga de cierto modo.

Sin embargo, la saturación de este medio suele incrementar en horas muy concurridas, debido al porcentaje tan alto de personas que lo usan (no únicamente estudiantes), lo que a su vez ocasiona conglomeraciones, por lo que ciertas rutas van a su capacidad máxima de pasajeros. Como consecuencia, el estudiante tiene la posibilidad de retrasarse. Otro de los problemas comunes relacionado a este es que hay una gran parte de estudiantes que no tienen una ruta directa o viven en fraccionamientos lejanos; por esta razón se tiene que tomar más de una ruta para llegar a su destino y, derivado de esto, se obtiene que el estudiante posiblemente se retrase.

1.2 Justificación del Proyecto

Para resolver estos problemas y eficientar el tiempo/transporte de ciertos estudiantes se desarrollará una solución que consiste en una plataforma de transporte compartido exclusiva para la comunidad universitaria.

1.3 Objetivo General y Alcance

Los estudiantes (conductores y pasajeros) registran sus horarios de clases, desde qué zona salen, hacia qué campus van y cuántos minutos de tolerancia tienen para esperar. El sistema cruza automáticamente todas estas variables y hace el emparejamiento, conectando a quienes tienen rutas y horarios compatibles para que compartan el viaje y los gastos de forma segura.

2. Funcionalidades principales

2.1.1 Sincronización y Emparejamiento por Horarios

La principal Diferencia radica en que no es un simple viaje aislado sino aquí los estudiantes tienen muchas más facilidades, alguna de las funcionalidades pensadas a implementar es la de cargar el horario de su carga académica o bien darle al usuario la facilidad de configurarlo manualmente en caso de errores o simple preferencia.

- **Conductor:** Registra la hora de salida de casa y termino de ultima clase
- **Pasajero:** Registra su hora de entrada y salida
- **Motor lógico:** Compara ambos horarios. Si el pasajero sale a la 1PM y el conductor a la 1:15PM, el sistema detecta una superposición viable para el viaje de regreso

2.1.2 Filtro de Tolerancia por Tiempo de Espera

Dado que no todos los horarios coinciden exactamente el usuario podrá filtrar por tiempo de espera al que está dispuesto esperar, esto amplía la coincidencia con otros usuarios.

- Si un conductor pasa por el fraccionamiento a las 6:30 AM, pero el pasajero prefiere salir a las 6:45 AM y configuró una tolerancia de 20 minutos, el sistema hace Match (porque 6:30 AM entra en el rango de tolerancia de espera).
- Si el pasajero solo está dispuesto a esperar 5 minutos, el viaje se cancela o bien busca coincidencias conforme a lo dado.

2.1.3 Filtro de Multi-Universidad y Campus

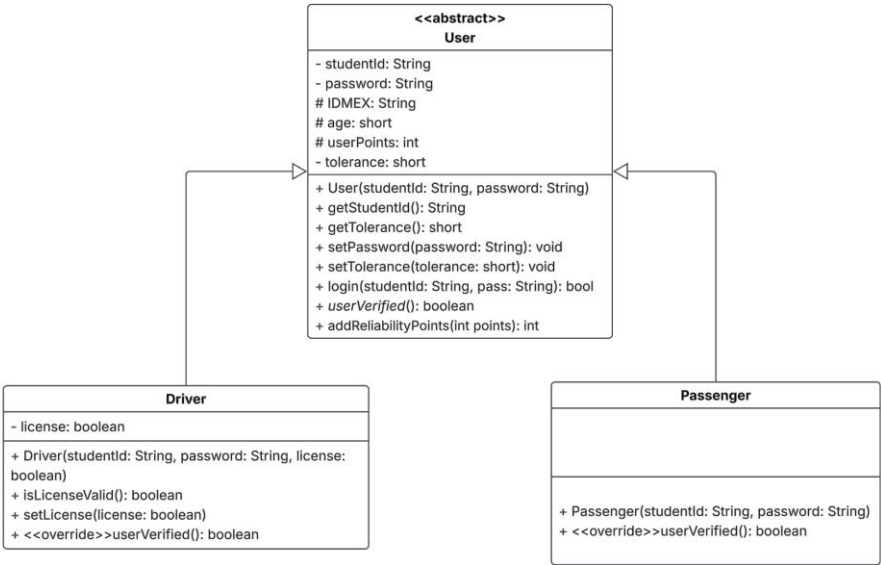
Un pasajero de la Facultad de Matemáticas puede hacer match con un conductor de la Facultad de Ingeniería dado que ambas se encuentran en el mismo campus o zona (Campus de Ciencias Exactas), optimizando el viaje para estudiantes de facultades vecinas.

2.1.5 Búsqueda de ruta más eficiente

La ruta consiste en un punto de partida y un punto de destino, se utilizará el algoritmo de Dijkstra (el algoritmo para encontrar el camino más corto entre dos nodos) para encontrar la ruta más eficiente y así evitar perjudicar al conductor.

3. Diseño de Arquitectura Orientada a Objetos

3.1 UML



3.2 Diccionario de clases

1. Clase Abstracta: User

Clase abstracta que sirve como base para encapsular la identidad del usuario y los parámetros generales de viaje.

Modificador de acceso	Elemento	Tipo de dato	Descripción
private(-)	studentId	String	Guarda la matrícula y nombre del usuario.
private(-)	password	String	Guarda la contraseña de forma encriptada para autenticación.

protected(#)	IDMEX	String	INE/Identificación guardados por motivos de seguridad institucional y reportes ante autoridades en caso de accidentes.
protected(#)	age	short	
protected(#)	userPoints	short	Puntos de confiabilidad del usuario.
private(-)	tolerance	short	Ventana de tiempo máximo que el usuario está dispuesto a esperar.
public(+)	User(studentId, password)	Constructor	
public(+)	getStudentid()	String	
public(+)	getTolerance()	String	
public(+)	setPassword(password)	void	
public(+)	setTolerance(tolerance)	void	
public(+)	login(studentid, password)	boolean	
public(+)	userVerified()	boolean	Método abstracto.
public(+)	addReliabilityPoints(points)	int	Modifica el puntaje de reputación del usuario.

2. Subclase: Driver (Herencia de User)

Representa la entidad del estudiante que provee el vehículo, registra sus horarios de trayecto y dispone asientos vacíos para compartir.

Modificador de acceso	Elemento	Tipo de dato	Descripción
private(-)	license	boolean	
public(+)	Driver(studentId, password, license)	Constructor	
public(+)	isLicenseValid()	boolean	
public(+)	setLicense(license)	void	
public(+)	userVerified()	Boolean	@Override. Implementación polimórfica que retorna true si IDMEX y license son válidos.

3. Subclase: Passenger (Herencia de User)

Representa la entidad del estudiante que solicita el viaje hacia o desde el campus basándose en sus horarios de clase.

Modificador de acceso	Elemento	Tipo de dato	Descripción
public(+)	Passenger(studentId, password)	Constructor	
public(+)	userVerified()	boolean	@Override. Implementación polimórfica que

			retorna true si IDMEX es válido.
--	--	--	-------------------------------------

4. Estrategia de Persistencia y Errores

Description of what data will be read/written to the .txt file and the list of custom exceptions we will implement. (TO DO)

5. Justificación de Principios SOLID

Explain how the design respects extensibility (for example, how the Matchmaking interface complies with the Open/Closed principle). (TO DO)